

**Gemini
Near
Infra-Red
Imager
and
Wavefront
Sensor**



Near IR Imager (NIRI) Software— Project History Document

Hubert Yamada

niri_h ty_010 / 02 / 1999-10-23

This report is a description of lessons learned while developing the NIRI software.

1 Introduction

This report is a summary of lessons learned during the process of developing the NIRI software.

This report was heavily influenced by [34].

1.1 Purpose of the document

This report is a summary of useful information that was learned during the development of the NIRI software. It is provided here as a guide to future projects that may choose to use NIRI as a basis for project development.

The intended audience of this document is:

- the Gemini instrument development community;
- the Gemini Controls group.

1.2 Scope

No software project will be developed from this document.

1.3 Definitions, Acronyms and Abbreviations

See “Near IR Imager (NIRI)—Definitions, Abbreviations, and References”.

1.4 Stylistic Conventions

References to documents in Section 1.5, “References,” on page 2 of “Near IR Imager (NIRI)—Definitions, Abbreviations, and References” are given like this; [1].

1.5 References

See “Near IR Imager (NIRI)—Definitions, Abbreviations, and References”.

2 Caveats

At this point in time, the NIRI software developers feel that NIRI, as implemented, is not a good choice as a starting point for other software projects. There are some severe flaws in the current design that should not be propagated to other instrument projects.

This document is not intended to be a criticism of the designers and implementers of the CICS and NIRI software. It is recognized that there were severe design constraints which were placed on them at an early stage, which forced some unwise design decisions. Although many of these constraints were later lifted, it was too late to correct the implementation of the software.

3 History

The software design for NIRI began in the early part of 1997; NIRI is expected to be delivered in late 1999. During this time, the design of NIRI has changed significantly for many reasons.

3.1 Early attempts

Originally, the NIRI developers were told that very little software development would be necessary. The CICS was a prototype system which would be provided for use as a template for the instrument, which could easily be customized to fit the NIRI mechanisms. It was to be the simple process of rearranging a few capfast diagrams, and adding links to existing records, to provide the actual mechanism control. At that time, the design for NIRI seemed quite straightforward. Very little time had been budgeted for software development, on the belief that the design would be trivial, based largely on an existing design. The implementation was also expected to be a relatively simple task, since it was expected to be a minor modification of an existing implementation.

At that early stage, the only task for the NIRI software developers was to learn EPICS, which appeared to be a relatively simple task. At that time, no attention was paid to the process of record development, indeed, to any of the mechanisms for embedding C code within an EPICS system. Simple EPICS databases were built. From this early experience, it seemed that building a mechanism-control database would be a fairly simple task.

A copy of the CICS was obtained fairly quickly. Unfortunately, this was not the final CICS release, nor would the final release be available for some time, because the CICS project was

experiencing unexpected delays. The CICS turned out to be quite a bit larger and more complicated than expected; however, at the time, this did not seem to be a major problem, because it was believed that it would not be necessary to deal with most of the complexity of the system. It was thought that the bulk of the implementation could be treated as “black boxes,” not requiring understanding of the details.

The first task seemed obvious: Customize the CICS to produce a working prototype for the NIRI system. It was felt that the working prototype would be updated when the final version of the CICS was provided, and it was believed that the prototype could trivially be modified to control actual motors. Although it was realized that this was an early release of the CICS, it was thought that it would be fairly easy to upgrade the system to a new CICS release, because the bulk of the implementation would be left untouched.

Unfortunately, it was found that the CICS was far more difficult to modify than expected. Weaknesses of capfast became obvious, and the extremely large number of connections required were found to be quite unwieldy. Furthermore, the existing diagrams had not been designed with room for expansion, requiring a significant amount of effort to increase the number of mechanisms from the small number that were present in the simulated database. Furthermore, the diagrams were extremely difficult to understand. It took considerable time to figure out precisely what each mechanism was doing. (In later releases, comments the CICS developers added comments, which reduced the amount of effort required to understand the diagram.) Ultimately, however, a working prototype for the NIRI mechanisms was completed, and the NIRI developers had developed a much better understanding of the way that the CICS worked.

Soon after, however, a new version of the CICS was released. This too, was not the final release. It was immediately obvious that there was a problem. Minor changes had been made throughout the entire system. A shortcoming of capfast suddenly became painfully visible: even the most trivial changes in a capfast diagram made significant changes in the underlying schematic files. Although the files were text-based, trying to compare the new version to the older version was fruitless. It was clear that the prototype would have to be re-implemented using the newer version of the CICS. Because this was not the final version of the CICS, it was felt that any time spent in developing a CICS-based prototype would be wasted, so work was temporarily suspended on the public interface to NIRI.

While the work on the prototype was taking place, early prototype mechanisms were being designed and fabricated, and motors were available for testing. Simple databases were created, and, in surprisingly short time, it was possible to create a database to control the motor using the stepper motor record. It was also quite easy to read out the values from the Hall effect sensors. This early success was to prove misleading, however, because further testing showed the stepper motor record to be quite unreliable. It took quite a bit of time to isolate the problems with the stepper motor record, and at the time the NIRI developers discovered far more about the inner workings of EPICS than was desired; though this knowledge would eventually turn out to be quite handy. The stepper motor record was never completely reliable, though.

At this point, an attempt was made to implement the proposed home finding algorithm. This required the Hall effect sensors to be read, while moving the motor. When the strongest signal was read from the sensor, that would be the motor home position. It was realized that a more sophisticated algorithm would be needed for the final system, but that this would be sufficient for an early prototype.

At this time, the NIRI developers were constrained by the requirement—verbally presented, but apparently mandatory, and not negotiable—that everything be done using capfast and the standard EPICS records. The subroutine record had explicitly been discouraged, though it might be allowed to be used. A significant amount of effort was expended in trying to create a search algorithm using these tools. Eventually, a diagram was created that was able to perform the necessary functions, to some extent. However, it was clear from the complexity of the system that implemented the simplest possible algorithm, that the effort required in implementing any refinements would be prohibitive.

At that time, the NIRI developers were still attempting to fulfil the all requirements, both written and verbal. Inspection of the CICS showed that the EPICS state notation language (SNL) was used extensively, so it was felt that SNL was a permissible development tool. Implementing a simple search algorithm proved to be relatively simple to implement. However any attempts to refine the algorithm quickly exposed weaknesses in SNL. In particular, the lack of simple ways to perform more than the most trivial loops was a constant source of aggravation, as was the poor support for code reuse. It was clear that implementing any sort of useful algorithm would require far more time and effort than was available, and that the resulting system would be completely inflexible and impossible to maintain.

3.2 The first working system.

At this point, it was clear that, as designed, the NIRI software would never be built. The available tools were simply insufficient, and frustration was building. Fortunately, at this time the attitude at Gemini had been changing. Experience with actual systems, and lack of progress on the part of the software groups had led to a more liberal attitude. The arbitrary restrictions on the use of the full EPICS tool set were quietly dropped, and we were informed that we could use anything that we felt was necessary, though Gemini did not want to see the entire NIRI control system embedded within a single subroutine record.

At that point, embedding the entire NIRI control system within a single subroutine record was a very tempting prospect, but with a certain amount of regret, it was decided to take a more moderate approach. A single gensub (generalized subroutine) record would be used to embed most of the control algorithms. This gensub record would be connected to the motor record which would control the motor and to analog input records which would read the Hall effect sensor values. It would contain a subroutine which would embed the necessary control functions. For the first time, it was actually possible for a NIRI mechanism to find its home position, using code that was somewhat maintainable.

Soon, it was realized that the gensub record was insufficient. The NIRI control algorithms required a fair amount of control over the output links, and the gensub record was not intended to provide this. Furthermore, there seemed to be no natural way to deal with slight variations between the different mechanisms, while taking advantage of the similarity between the mechanisms.

To get around these problems, it was decided to create a new, custom EPICS record, the hall-step record, which was based on the gensub record. Modifying the code within the record provided direct control over the outputs of the record. Different device types were defined; each device type would control the idiosyncrasies of each class of mechanism. The hallstep record would provide a common interface to EPICS so that all mechanisms could be controlled in the same way. The hallstep record was extended so that it could do a variety of tasks, not just finding the home position. Eventually, the hallstep record became the control

point for the mechanism, containing all of the more complicated looping and testing functions. A prototype control system was created which was capable of controlling all of the different mechanisms that were within NIRI. A mechanism simulator was written, which made it possible to build a complete, functional NIRI control system which was capable of controlling two prototype mechanisms and a full compliment of simulated mechanisms.

The problems with the stepper motor record continued to be a constant source of frustration, and time was not available to fix its problems. Fortunately, at this time, we were provided with a copy of the motor record. Unfortunately, the motor record was not a simple drop-in replacement for the stepper motor record, but modifying the existing system to use the motor record instead of the stepper motor record was a relatively minor task.

The process of replacing the stepper motor record as well as the process of refining the main control diagram pointed out a serious weakness in the current system. There was a separate capfast diagram to control each mechanism type. Even the slightest change in a capfast diagram had to be made manually in each diagram. It was realized that proper use of capfast definitions would make it possible to replace multiple diagrams with a single diagram, which would simplify the task of maintenance and modification. Although some symbols would be unused for particular mechanisms, the gain in efficiency far outweighed the slight cost of an unused symbol.

3.3 Integration with the CICS

The Core Instrument Control system (the CICS) was then integrated with the NIRI software, with the assistance of Steven Beard. This integration was a very smooth process, requiring only minor modifications, with the CICS and the NIRI software. This was a source of great comfort for the NIRI and CICS designers, and it was felt that the remaining work for NIRI was to refine the control algorithms to work with each actual working mechanisms. This early optimism was soon to be found to be unjustified.

It was very quickly realized that the NIRI/CICS system took an excessive amount of time to boot. This was a serious problem, and there was no way to overcome it, without completely rewriting the CICS or replacing it. In order to maintain a reasonable level of productivity, the NIRI developers chose to continue developing the NIRI software with minimal testing of the NIRI/CICS integrated system. This was, on the whole, a good decision, and significantly increased the productivity of the NIRI software. Unfortunately, it also caused several interface issues to be postponed, which caused a significant underestimate of the time required to finish the system.

3.4 The interlock system

At this point, working mechanisms were becoming available, and a great deal of time and effort were devoted to getting the mechanism control refined and to verify correct operation of the mechanisms. At this point, it was felt that the decision to postpone development of the overall system was a sound one. Unfortunately, this meant that several user interface elements had to be built into the NIRI low-level support system. Various interlocks to prevent operation of the motors at critical times were added (partly at the request of the GNIRS developers). It was felt that the interlocks should be implemented in the lower-level code, so that the implementation would be simple to verify by inspection.

Getting the interlocks to work with the CICS code turned out to be an unexpected challenge. Since there was no intrinsic support for state variables in the CICS, it was felt that the easiest approach was to treat the interlock variables as a simple two-position mechanism. Since the interlocks were all interrelated, it was felt that the interlocks could easily be treated as a four axis mechanism. It was realized that this approach would lead to a more complicated implementation, but would permit large amounts of the CICS code to be used unmodified. In retrospect, this proved to be a very bad decision, possibly the worst decision that the NIRI developers made on the project.

Expanding the existing two-axis discreet mechanism to support a four-axis mechanism was found to be rather challenging. It required a surprising amount of rearrangement of the existing diagrams, to add the new axes. Furthermore, in order to support all of the fields that were required by the SNL code, it was necessary to add a large number of dummy field, including a completion field which would toggle immediately (since the flags could be set instantly).

Ultimately, when it became necessary to make major modifications to the mechanism-control (described below) it was decided that it was a waste of effort to make the same changes to the pseudo mechanisms. At that point, the code was copied (unfortunately, no simple way to share common code was found), and the supporting code for the interlocks was simplified.

The apparent simplifications that arose from making a flag look like a mechanism turned out to be illusory, and actually ended up wasting far more time and effort than it saved.

3.5 The NIRI temperature control system

The low level control system was made to function by writing a custom record to interface to the off-the-shelf temperature controllers. The mechanisms were supported using a custom record, which communicated with the built-in serial ports on the CPU and a XYCOM serial I/O interface board, which was chosen because of previous experience with the board. The existing EPICS serial control software was not used, because the control task was quite simple. In retrospect, some time might have been saved by using the existing software, and choosing a board with existing EPICS support.

The NIRI temperature control system had many of the problems in interfacing to the CICS that the interlock system had. The initial attempt at controlling the system was made by treating the temperature control as a three axis mechanism. The difficulties which plagued the interlock system also plagued the temperature control system, and ultimately, the temperature control system was unable to share much code with the mechanism control system; most of the capfast diagrams had to be replicated and customized.

3.6 The NIRI diagnostic functions

3.7 The NIRI verification functions

4 CICS

4.1 Successful elements of the CICS

The CICS provides all of the necessary ICD compliant external interfaces.

4.2 Problems with the CICS

The CICS is very slow to reboot, and difficult to modify. There are a large number of nearly-identical diagrams, all of which must be modified in order to make any changes to the system. It assumes that all mechanisms are handled in an identical fashion, and requires a large number of diagrams to be duplicated if one mechanism must be handled differently.

5 The NIRI software

5.1 Successful elements of NIRI

The strong separation of the higher-level interface from the low-level mechanism-control database has turned out to be very useful. This has made it possible to rapidly develop customized mechanism controls, without worries about meeting interface specifications. It has streamlined the development cycle, by reducing the time wasted waiting for the system to reboot. It made it possible to replace the mechanism implementation with a simulation, so that the public interfaces could be tested and debugged long before the actual mechanisms were available.

Using identical capfast diagrams for all mechanisms has turned out to be a tremendous time saver.

Using the same custom record for all mechanisms, with custom device support for the different device types has been quite useful in reducing the amount of work involved. Being able to replace the drivers with simulated mechanisms has insured that the simulation is able to reproduce the working system with a high degree of accuracy. This has allowed much development and debugging to be carried out before the mechanisms were available.

5.2 Problems with NIRI

6 Lessons learned

- If a template is to be provided, the initial implementation should be completed before any of the software which is to be derived from it is complete.
- Time should be allocated to re-implement the template after experience with working systems.
- Time should be allocated to re-implement the working systems, after the template has been modified.
- If the template is complicated, then the working systems will be very complicated.
- If the template is difficult to maintain, then the working systems will be impossible to maintain.
- Arbitrarily restricting developers to a subset of the available tools can lead to overly complicated designs.
- Be careful about scaling problems. Designs that work for simple test systems can become unmanageable for larger systems.
- It is better to have unnecessary symbols in a diagram than to have multiple nearly-identical diagrams. Proper use of definitions can combine nearly-identical diagrams.

7 Suggestions

8 Conclusion

At this point, the NIRI software should be considered a qualified success, in that it is capable of controlling the instrument and providing the required functions. However, the implementation is considerably more complicated than it needs to be, and is quite difficult to modify. The time that the system takes to reboot is a concern, because it makes it quite time consuming to fix even the smallest bugs. Insufficient time was available to refine many of the algorithms, particularly the algorithms which are used to find the home position.